```
; Listing 2.
; ================================================
; CamelForth for the Zilog Z80
; (c) 1994 Bradford J. Rodriguez
; Permission is granted to freely copy, modify,
; and distribute this program for personal or
; educational use.  Commercial inquiries should
; be directed to the author at 221 King St. E.,
; #32, Hamilton, Ontario L8N 1B5 Canada
;
; CAMEL80.AZM: Code Primitives
;   Source code is for the Z80MR macro assembler.
;   Forth words are documented as follows:
;x   NAME     stack -- stack    description
;   where x=C for ANS Forth Core words, X for ANS
;   Extensions, Z for internal or private words.
;
; Direct-Threaded Forth model for Zilog Z80
; 16 bit cell, 8 bit char, 8 bit (byte) adrs unit
;    Z80 BC = Forth TOS (top Param Stack item)
;        HL =       W    working register
;        DE =       IP   Interpreter Pointer
;        SP =       PSP  Param Stack Pointer
;        IX =       RSP  Return Stack Pointer
;        IY =       UP   User area Pointer
;    A, alternate register set = temporaries
;
; Revision history:
;   19 Aug 94 v1.0
;   25 Jan 95 v1.01  now using BDOS function 0Ah
;       for interpreter input; TIB at 82h.
;   02 Mar 95 v1.02  changed ALIGN to ALIGNED in
;       S" (S"); changed ,BRANCH to ,XT in DO.
; ================================================
; Macros to define Forth headers
; HEAD  label,length,name,action
; IMMED label,length,name,action
;    label  = assembler name for this word
;             (special characters not allowed)
;    length = length of name field
;    name   = Forth's name for this word
;    action = code routine for this word, e.g.
;             DOCOLON, or DOCODE for code words
; IMMED defines a header for an IMMEDIATE word.
;
DOCODE  EQU 0      ; flag to indicate CODE words
link    DEFL 0     ; link to previous Forth word

head    MACRO   #label,#length,#name,#action
        DW link
        DB 0
link    DEFL $
        DB #length,'#name'
#label:
        IF  .NOT.(#action=DOCODE)
        call #action
        ENDIF
        ENDM

immed   MACRO   #label,#length,#name,#action
        DW link
        DB 1
link    DEFL $
        DB #length,'#name'
#label:
        IF  .NOT.(#action=DOCODE)
        call #action
        ENDIF
        ENDM
```

```
; The NEXT macro (7 bytes) assembles the 'next'
; code in-line in every Z80 CamelForth CODE word.
next    MACRO
        ex de,hl
        ld e,(hl)
        inc hl
        ld d,(hl)
        inc hl
        ex de,hl
        jp (hl)
        ENDM

; NEXTHL is used when the IP is already in HL.
nexthl  MACRO
        ld e,(hl)
        inc hl
        ld d,(hl)
        inc hl
        ex de,hl
        jp (hl)
        ENDM

; RESET AND INTERRUPT VECTORS ==================
; ...are not used in the CP/M implementation
; Instead, we have the...

; CP/M ENTRY POINT
        org 100h
reset:  ld hl,(6h)   ; BDOS address, rounded down
        ld l,0       ;    = end of avail.mem (EM)
        dec h        ; EM-100h
        ld sp,hl     ;      = top of param stack
        inc h        ; EM
        push hl
        pop ix       ;      = top of return stack
        dec h        ; EM-200h
        dec h
        push hl
        pop iy       ;      = bottom of user area
        ld de,1      ; do reset if COLD returns
        jp COLD      ; enter top-level Forth word

; Memory map:
;   0080h       Terminal Input Buffer, 128 bytes
;   0100h       Forth kernel = start of CP/M TPA
;     ? h       Forth dictionary (user RAM)
;   EM-200h     User area, 128 bytes
;   EM-180h     Parameter stack, 128B, grows down
;   EM-100h     HOLD area, 40 bytes, grows down
;   EM-0D8h     PAD buffer, 88 bytes
;   EM-80h      Return stack, 128 B, grows down
;   EM          End of RAM = start of CP/M BDOS
; See also the definitions of U0, S0, and R0
; in the "system variables & constants" area.
; A task w/o terminal input requires 200h bytes.
; Double all except TIB and PAD for 32-bit CPUs.

; INTERPRETER LOGIC ============================
; See also "defining words" at end of this file

;C EXIT     --      exit a colon definition
    head EXIT,4,EXIT,docode
        ld e,(ix+0)    ; pop old IP from ret stk
        inc ix
        ld d,(ix+0)
        inc ix
        next
```

```
;Z lit      -- x    fetch inline literal to stack
; This is the primtive compiled by LITERAL.
    head lit,3,lit,docode
        push bc         ; push old TOS
        ld a,(de)       ; fetch cell at IP to TOS,
        ld c,a          ;        advancing IP
        inc de
        ld a,(de)
        ld b,a
        inc de
        next

;C EXECUTE   i*x xt -- j*x   execute Forth word
;C                           at 'xt'
    head EXECUTE,7,EXECUTE,docode
        ld h,b          ; address of word -> HL
        ld l,c
        pop bc          ; get new TOS
        jp (hl)         ; go do Forth word

; DEFINING WORDS ================================

; ENTER, a.k.a. DOCOLON, entered by CALL ENTER
; to enter a new high-level thread (colon def'n.)
; (internal code fragment, not a Forth word)
; N.B.: DOCOLON must be defined before any
; appearance of 'docolon' in a 'word' macro!
docolon:                ; (alternate name)
enter:  dec ix          ; push old IP on ret stack
        ld (ix+0),d
        dec ix
        ld (ix+0),e
        pop hl          ; param field adrs -> IP
        nexthl          ; use the faster 'nexthl'

;C VARIABLE   --      define a Forth variable
;   CREATE 1 CELLS ALLOT ;
; Action of RAM variable is identical to CREATE,
; so we don't need a DOES> clause to change it.
    head VARIABLE,8,VARIABLE,docolon
        DW CREATE,LIT,1,CELLS,ALLOT,EXIT
; DOVAR, code action of VARIABLE, entered by CALL
; DOCREATE, code action of newly created words
docreate:
dovar:  ; -- a-addr
        pop hl      ; parameter field address
        push bc     ; push old TOS
        ld b,h      ; pfa = variable's adrs -> TOS
        ld c,l
        next

;C CONSTANT   n --      define a Forth constant
;   CREATE , DOES> (machine code fragment)
    head CONSTANT,8,CONSTANT,docolon
        DW CREATE,COMMA,XDOES
; DOCON, code action of CONSTANT,
; entered by CALL DOCON
docon:  ; -- x
        pop hl      ; parameter field address
        push bc     ; push old TOS
        ld c,(hl)   ; fetch contents of parameter
        inc hl      ;    field -> TOS
        ld b,(hl)
        next

;Z USER     n --        define user variable 'n'
;   CREATE , DOES> (machine code fragment)
    head USER,4,USER,docolon
        DW CREATE,COMMA,XDOES
```

```
; DOUSER, code action of USER,
; entered by CALL DOUSER
douser:  ; -- a-addr
        pop hl     ; parameter field address
        push bc    ; push old TOS
        ld c,(hl)  ; fetch contents of parameter
        inc hl     ;    field
        ld b,(hl)
        push iy    ; copy user base address to HL
        pop hl
        add hl,bc  ;    and add offset
        ld b,h     ; put result in TOS
        ld c,l
        next

; DODOES, code action of DOES> clause
; entered by       CALL fragment
;                  parameter field
;                       ...
;        fragment: CALL DODOES
;                  high-level thread
; Enters high-level thread with address of
; parameter field on top of stack.
; (internal code fragment, not a Forth word)
dodoes: ; -- a-addr
        dec ix         ; push old IP on ret stk
        ld (ix+0),d
        dec ix
        ld (ix+0),e
        pop de         ; adrs of new thread -> IP
        pop hl         ; adrs of parameter field
        push bc        ; push old TOS onto stack
        ld b,h         ; pfa -> new TOS
        ld c,l
        next

; CP/M TERMINAL I/O ============================
cpmbdos EQU 5h          ; CP/M BDOS entry point

;Z BDOS   de c -- a   call CP/M BDOS
    head BDOS,4,BDOS,docode
        ex de,hl    ; save important Forth regs
        pop de      ;  (DE,IX,IY) & pop DE value
        push hl
        push ix
        push iy
        call cpmbdos
        ld c,a      ; result in TOS
        ld b,0
        pop iy      ; restore Forth regs
        pop ix
        pop de
        next

;C EMIT     c --    output character to console
;   6 BDOS DROP ;
; warning: if c=0ffh, will read one keypress
    head EMIT,4,EMIT,docolon
        DW LIT,06H,BDOS,DROP,EXIT

;Z SAVEKEY  -- addr  temporary storage for KEY?
    head savekey,7,SAVEKEY,dovar
        DW 0

;X KEY?     -- f    return true if char waiting
;   0FF 6 BDOS DUP SAVEKEY C! ;   rtns 0 or key
; must use BDOS function 6 to work with KEY
    head querykey,4,KEY?,docolon
        DW LIT,0FFH,LIT,06H,BDOS
```

```
        DW DUP,SAVEKEY,CSTORE,EXIT

;C KEY      -- c    get character from keyboard
;   BEGIN SAVEKEY C@ 0= WHILE KEY? DROP REPEAT
;   SAVEKEY C@  0 SAVEKEY C! ;
; must use CP/M direct console I/O to avoid echo
; (BDOS function 6, contained within KEY?)
    head KEY,3,KEY,docolon
KEY1:   DW SAVEKEY,CFETCH,ZEROEQUAL,qbranch,KEY2
        DW QUERYKEY,DROP,branch,KEY1
KEY2:   DW SAVEKEY,CFETCH,LIT,0,SAVEKEY,CSTORE
        DW EXIT

;Z CPMACCEPT  c-addr +n -- +n'  get line of input
;   SWAP 2 - TUCK C!      max # of characters
;   DUP 0A BDOS DROP      CP/M Get Console Buffer
;   1+ C@  0A EMIT ;      get returned count
; Note: requires the two locations before c-addr
; to be available for use.
    head CPMACCEPT,9,CPMACCEPT,docolon
        DW SWOP,LIT,2,MINUS,TUCK,CSTORE
        DW DUP,LIT,0Ah,BDOS,DROP
        DW ONEPLUS,CFETCH,LIT,0Ah,EMIT,EXIT

;X BYE     i*x --    return to CP/M
    head bye,3,bye,docode
        jp 0

; STACK OPERATIONS ==============================

;C DUP      x -- x x      duplicate top of stack
    head DUP,3,DUP,docode
pushtos: push bc
        next

;C ?DUP     x -- 0 | x x    DUP if nonzero
    head QDUP,4,?DUP,docode
        ld a,b
        or c
        jr nz,pushtos
        next

;C DROP     x --          drop top of stack
    head DROP,4,DROP,docode
poptos: pop bc
        next

;C SWAP     x1 x2 -- x2 x1    swap top two items
    head SWOP,4,SWAP,docode
        pop hl
        push bc
        ld b,h
        ld c,l
        next

;C OVER    x1 x2 -- x1 x2 x1   per stack diagram
    head OVER,4,OVER,docode
        pop hl
        push hl
        push bc
        ld b,h
        ld c,l
        next

;C ROT    x1 x2 x3 -- x2 x3 x1  per stack diagram
    head ROT,3,ROT,docode
        ; x3 is in TOS
        pop hl          ; x2
        ex (sp),hl      ; x2 on stack, x1 in hl
```

```
        push bc
        ld b,h
        ld c,l
        next

;X NIP    x1 x2 -- x2           per stack diagram
    head NIP,3,NIP,docolon
        DW SWOP,DROP,EXIT

;X TUCK   x1 x2 -- x2 x1 x2     per stack diagram
    head TUCK,4,TUCK,docolon
        DW SWOP,OVER,EXIT

;C >R    x --   R: -- x   push to return stack
    head TOR,2,>R,docode
        dec ix          ; push TOS onto rtn stk
        ld (ix+0),b
        dec ix
        ld (ix+0),c
        pop bc          ; pop new TOS
        next

;C R>    -- x    R: x --   pop from return stack
    head RFROM,2,R>,docode
        push bc         ; push old TOS
        ld c,(ix+0)     ; pop top rtn stk item
        inc ix          ;       to TOS
        ld b,(ix+0)
        inc ix
        next

;C R@    -- x     R: x -- x   fetch from rtn stk
    head RFETCH,2,R@,docode
        push bc         ; push old TOS
        ld c,(ix+0)     ; fetch top rtn stk item
        ld b,(ix+1)     ;       to TOS
        next

;Z SP@  -- a-addr       get data stack pointer
    head SPFETCH,3,SP@,docode
        push bc
        ld hl,0
        add hl,sp
        ld b,h
        ld c,l
        next

;Z SP!  a-addr --       set data stack pointer
    head SPSTORE,3,SP!,docode
        ld h,b
        ld l,c
        ld sp,hl
        pop bc          ; get new TOS
        next

;Z RP@  -- a-addr       get return stack pointer
    head RPFETCH,3,RP@,docode
        push bc
        push ix
        pop bc
        next

;Z RP!  a-addr --       set return stack pointer
    head RPSTORE,3,RP!,docode
        push bc
        pop ix
        pop bc
        next
```

```
; MEMORY AND I/O OPERATIONS ====================

;C !        x a-addr --   store cell in memory
    head STORE,1,!,docode
        ld h,b          ; address in hl
        ld l,c
        pop bc          ; data in bc
        ld (hl),c
        inc hl
        ld (hl),b
        pop bc          ; pop new TOS
        next

;C C!      char c-addr --    store char in memory
    head CSTORE,2,C!,docode
        ld h,b          ; address in hl
        ld l,c
        pop bc          ; data in bc
        ld (hl),c
        pop bc          ; pop new TOS
        next

;C @       a-addr -- x   fetch cell from memory
    head FETCH,1,@,docode
        ld h,b          ; address in hl
        ld l,c
        ld c,(hl)
        inc hl
        ld b,(hl)
        next

;C C@     c-addr -- char   fetch char from memory
    head CFETCH,2,C@,docode
        ld a,(bc)
        ld c,a
        ld b,0
        next

;Z PC!     char c-addr --    output char to port
    head PCSTORE,3,PC!,docode
        pop hl          ; char in L
        out (c),l       ; to port (BC)
        pop bc          ; pop new TOS
        next

;Z PC@     c-addr -- char   input char from port
    head PCFETCH,3,PC@,docode
        in c,(c)        ; read port (BC) to C
        ld b,0
        next

; ARITHMETIC AND LOGICAL OPERATIONS =============

;C +       n1/u1 n2/u2 -- n3/u3     add n1+n2
    head PLUS,1,+,docode
        pop hl
        add hl,bc
        ld b,h
        ld c,l
        next

;X M+       d n -- d         add single to double
    head MPLUS,2,M+,docode
        ex de,hl
        pop de          ; hi cell
        ex (sp),hl      ; lo cell, save IP
        add hl,bc
        ld b,d          ; hi result in BC (TOS)
        ld c,e
```

```
        jr nc,mplus1
        inc bc
mplus1: pop de          ; restore saved IP
        push hl         ; push lo result
        next

;C -      n1/u1 n2/u2 -- n3/u3    subtract n1-n2
    head MINUS,1,-,docode
        pop hl
        or a
        sbc hl,bc
        ld b,h
        ld c,l
        next

;C AND    x1 x2 -- x3            logical AND
    head AND,3,AND,docode
        pop hl
        ld a,b
        and h
        ld b,a
        ld a,c
        and l
        ld c,a
        next

;C OR     x1 x2 -- x3           logical OR
    head OR,2,OR,docode
        pop hl
        ld a,b
        or h
        ld b,a
        ld a,c
        or l
        ld c,a
        next

;C XOR    x1 x2 -- x3            logical XOR
    head XOR,3,XOR,docode
        pop hl
        ld a,b
        xor h
        ld b,a
        ld a,c
        xor l
        ld c,a
        next

;C INVERT   x1 -- x2            bitwise inversion
    head INVERT,6,INVERT,docode
        ld a,b
        cpl
        ld b,a
        ld a,c
        cpl
        ld c,a
        next

;C NEGATE   x1 -- x2            two's complement
    head NEGATE,6,NEGATE,docode
        ld a,b
        cpl
        ld b,a
        ld a,c
        cpl
        ld c,a
        inc bc
        next
```

```
  ;C 1+      n1/u1 -- n2/u2       add 1 to TOS
      head ONEPLUS,2,1+,docode
          inc bc
          next

  ;C 1-      n1/u1 -- n2/u2     subtract 1 from TOS
      head ONEMINUS,2,1-,docode
          dec bc
          next

  ;Z ><      x1 -- x2         swap bytes (not ANSI)
      head swapbytes,2,><,docode
          ld a,b
          ld b,c
          ld c,a
          next

  ;C 2*      x1 -- x2         arithmetic left shift
      head TWOSTAR,2,2*,docode
          sla c
          rl b
          next

  ;C 2/      x1 -- x2        arithmetic right shift
      head TWOSLASH,2,2/,docode
          sra b
          rr c
          next

  ;C LSHIFT  x1 u -- x2    logical L shift u places
      head LSHIFT,6,LSHIFT,docode
          ld b,c        ; b = loop counter
          pop hl        ;   NB: hi 8 bits ignored!
          inc b         ; test for counter=0 case
          jr lsh2
  lsh1:   add hl,hl     ; left shift HL, n times
  lsh2:   djnz lsh1
          ld b,h        ; result is new TOS
          ld c,l
          next

  ;C RSHIFT  x1 u -- x2    logical R shift u places
      head RSHIFT,6,RSHIFT,docode
          ld b,c        ; b = loop counter
          pop hl        ;   NB: hi 8 bits ignored!
          inc b         ; test for counter=0 case
          jr rsh2
  rsh1:   srl h         ; right shift HL, n times
          rr l
  rsh2:   djnz rsh1
          ld b,h        ; result is new TOS
          ld c,l
          next

  ;C +!     n/u a-addr --       add cell to memory
      head PLUSSTORE,2,+!,docode
          pop hl
          ld a,(bc)       ; low byte
          add a,l
          ld (bc),a
          inc bc
          ld a,(bc)       ; high byte
          adc a,h
          ld (bc),a
          pop bc          ; pop new TOS
          next

  ; COMPARISON OPERATIONS ========================
```

```
;C 0=     n/u -- flag    return true if TOS=0
    head ZEROEQUAL,2,0=,docode
        ld a,b
        or c            ; result=0 if bc was 0
        sub 1           ; cy set   if bc was 0
        sbc a,a         ; propagate cy through A
        ld b,a          ; put 0000 or FFFF in TOS
        ld c,a
        next

;C 0<     n -- flag      true if TOS negative
    head ZEROLESS,2,0<,docode
        sla b           ; sign bit -> cy flag
        sbc a,a         ; propagate cy through A
        ld b,a          ; put 0000 or FFFF in TOS
        ld c,a
        next

;C =      x1 x2 -- flag         test x1=x2
    head EQUAL,1,=,docode
        pop hl
        or a
        sbc hl,bc       ; x1-x2 in HL, SZVC valid
        jr z,tostrue
tosfalse: ld bc,0
        next

;X <>     x1 x2 -- flag    test not eq (not ANSI)
    head NOTEQUAL,2,<>,docolon
        DW EQUAL,ZEROEQUAL,EXIT

;C <      n1 n2 -- flag        test n1<n2, signed
    head LESS,1,<,docode
        pop hl
        or a
        sbc hl,bc       ; n1-n2 in HL, SZVC valid
; if result negative & not OV, n1<n2
; neg. & OV => n1 +ve, n2 -ve, rslt -ve, so n1>n2
; if result positive & not OV, n1>=n2
; pos. & OV => n1 -ve, n2 +ve, rslt +ve, so n1<n2
; thus OV reverses the sense of the sign bit
        jp pe,revsense  ; if OV, use rev. sense
        jp p,tosfalse   ;   if +ve, result false
tostrue: ld bc,0ffffh   ;   if -ve, result true
        next
revsense: jp m,tosfalse ; OV: if -ve, reslt false
        jr tostrue      ;     if +ve, result true

;C >     n1 n2 -- flag         test n1>n2, signed
    head GREATER,1,>,docolon
        DW SWOP,LESS,EXIT

;C U<    u1 u2 -- flag       test u1<n2, unsigned
    head ULESS,2,U<,docode
        pop hl
        or a
        sbc hl,bc       ; u1-u2 in HL, SZVC valid
        sbc a,a         ; propagate cy through A
        ld b,a          ; put 0000 or FFFF in TOS
        ld c,a
        next

;X U>    u1 u2 -- flag     u1>u2 unsgd (not ANSI)
    head UGREATER,2,U>,docolon
        DW SWOP,ULESS,EXIT

; LOOP AND BRANCH OPERATIONS ====================

;Z branch   --                  branch always
```

```
    head branch,6,branch,docode
dobranch: ld a,(de)     ; get inline value => IP
        ld l,a
        inc de
        ld a,(de)
        ld h,a
        nexthl

;Z ?branch   x --              branch if TOS zero
    head qbranch,7,?branch,docode
        ld a,b
        or c            ; test old TOS
        pop bc          ; pop new TOS
        jr z,dobranch   ; if old TOS=0, branch
        inc de          ; else skip inline value
        inc de
        next

;Z (do)    n1|u1 n2|u2 --  R: -- sys1 sys2
;Z                          run-time code for DO
; '83 and ANSI standard loops terminate when the
; boundary of limit-1 and limit is crossed, in
; either direction.  This can be conveniently
; implemented by making the limit 8000h, so that
; arithmetic overflow logic can detect crossing.
; I learned this trick from Laxen & Perry F83.
; fudge factor = 8000h-limit, to be added to
; the start value.
    head xdo,4,(do),docode
        ex de,hl
        ex (sp),hl   ; IP on stack, limit in HL
        ex de,hl
        ld hl,8000h
        or a
        sbc hl,de    ; 8000-limit in HL
        dec ix       ; push this fudge factor
        ld (ix+0),h  ;    onto return stack
        dec ix       ;    for later use by 'I'
        ld (ix+0),l
        add hl,bc    ; add fudge to start value
        dec ix       ; push adjusted start value
        ld (ix+0),h  ;    onto return stack
        dec ix       ;    as the loop index.
        ld (ix+0),l
        pop de       ; restore the saved IP
        pop bc       ; pop new TOS
        next

;Z (loop)   R: sys1 sys2 --  | sys1 sys2
;Z                          run-time code for LOOP
; Add 1 to the loop index.  If loop terminates,
; clean up the return stack and skip the branch.
; Else take the inline branch.  Note that LOOP
; terminates when index=8000h.
    head xloop,6,(loop),docode
        exx
        ld bc,1
looptst: ld l,(ix+0)  ; get the loop index
        ld h,(ix+1)
        or a
        adc hl,bc    ; increment w/overflow test
        jp pe,loopterm  ; overflow=loop done
        ; continue the loop
        ld (ix+0),l  ; save the updated index
        ld (ix+1),h
        exx
        jr dobranch  ; take the inline branch
loopterm: ; terminate the loop
        ld bc,4      ; discard the loop info
```

```
        add ix,bc
        exx
        inc de       ; skip the inline branch
        inc de
        next

;Z (+loop)   n --   R: sys1 sys2 --  | sys1 sys2
;Z                      run-time code for +LOOP
; Add n to the loop index.  If loop terminates,
; clean up the return stack and skip the branch.
; Else take the inline branch.
    head xplusloop,7,(+loop),docode
        pop hl      ; this will be the new TOS
        push bc
        ld b,h
        ld c,l
        exx
        pop bc      ; old TOS = loop increment
        jr looptst

;C I        -- n   R: sys1 sys2 -- sys1 sys2
;C                  get the innermost loop index
    head II,1,I,docode
        push bc     ; push old TOS
        ld l,(ix+0) ; get current loop index
        ld h,(ix+1)
        ld c,(ix+2) ; get fudge factor
        ld b,(ix+3)
        or a
        sbc hl,bc   ; subtract fudge factor,
        ld b,h      ;   returning true index
        ld c,l
        next

;C J        -- n   R: 4*sys -- 4*sys
;C                  get the second loop index
    head JJ,1,J,docode
        push bc     ; push old TOS
        ld l,(ix+4) ; get current loop index
        ld h,(ix+5)
        ld c,(ix+6) ; get fudge factor
        ld b,(ix+7)
        or a
        sbc hl,bc   ; subtract fudge factor,
        ld b,h      ;   returning true index
        ld c,l
        next

;C UNLOOP   --   R: sys1 sys2 --  drop loop parms
    head UNLOOP,6,UNLOOP,docode
        inc ix
        inc ix
        inc ix
        inc ix
        next

; MULTIPLY AND DIVIDE ============================

;C UM*     u1 u2 -- ud   unsigned 16x16->32 mult.
    head UMSTAR,3,UM*,docode
        push bc
        exx
        pop bc      ; u2 in BC
        pop de      ; u1 in DE
        ld hl,0     ; result will be in HLDE
        ld a,17     ; loop counter
        or a        ; clear cy
umloop: rr h
        rr l
```

```
        rr d
        rr e
        jr nc,noadd
        add hl,bc
noadd:  dec a
        jr nz,umloop
        push de     ; lo result
        push hl     ; hi result
        exx
        pop bc      ; put TOS back in BC
        next

;C UM/MOD   ud u1 -- u2 u3   unsigned 32/16->16
    head UMSLASHMOD,6,UM/MOD,docode
        push bc
        exx
        pop bc      ; BC = divisor
        pop hl      ; HLDE = dividend
        pop de
        ld a,16     ; loop counter
        sla e
        rl d        ; hi bit DE -> carry
udloop: adc hl,hl   ; rot left w/ carry
        jr nc,udiv3
        ; case 1: 17 bit, cy:HL = 1xxxx
        or a        ; we know we can subtract
        sbc hl,bc
        or a        ; clear cy to indicate sub ok
        jr udiv4
        ; case 2: 16 bit, cy:HL = 0xxxx
udiv3:  sbc hl,bc   ; try the subtract
        jr nc,udiv4 ; if no cy, subtract ok
        add hl,bc   ; else cancel the subtract
        scf         ;   and set cy to indicate
udiv4:  rl e        ; rotate result bit into DE,
        rl d        ; and next bit of DE into cy
        dec a
        jr nz,udloop
        ; now have complemented quotient in DE,
        ; and remainder in HL
        ld a,d
        cpl
        ld b,a
        ld a,e
        cpl
        ld c,a
        push hl     ; push remainder
        push bc
        exx
        pop bc      ; quotient remains in TOS
        next

; BLOCK AND STRING OPERATIONS ==================

;C FILL   c-addr u char --  fill memory with char
    head FILL,4,FILL,docode
        ld a,c          ; character in a
        exx             ; use alt. register set
        pop bc          ; count in bc
        pop de          ; address in de
        or a            ; clear carry flag
        ld hl,0ffffh
        adc hl,bc       ; test for count=0 or 1
        jr nc,filldone  ;   no cy: count=0, skip
        ld (de),a       ; fill first byte
        jr z,filldone   ;   zero, count=1, done
        dec bc          ; else adjust count,
        ld h,d          ;   let hl = start adrs,
        ld l,e
```

```
        inc de          ;   let de = start adrs+1
        ldir            ;   copy (hl)->(de)
filldone: exx           ; back to main reg set
        pop bc          ; pop new TOS
        next

;X CMOVE   c-addr1 c-addr2 u --  move from bottom
; as defined in the ANSI optional String word set
; On byte machines, CMOVE and CMOVE> are logical
; factors of MOVE.  They are easy to implement on
; CPUs which have a block-move instruction.
    head CMOVE,5,CMOVE,docode
        push bc
        exx
        pop bc      ; count
        pop de      ; destination adrs
        pop hl      ; source adrs
        ld a,b      ; test for count=0
        or c
        jr z,cmovedone
        ldir        ; move from bottom to top
cmovedone: exx
        pop bc      ; pop new TOS
        next

;X CMOVE>  c-addr1 c-addr2 u --  move from top
; as defined in the ANSI optional String word set
    head CMOVEUP,6,CMOVE>,docode
        push bc
        exx
        pop bc      ; count
        pop hl      ; destination adrs
        pop de      ; source adrs
        ld a,b      ; test for count=0
        or c
        jr z,umovedone
        add hl,bc   ; last byte in destination
        dec hl
        ex de,hl
        add hl,bc   ; last byte in source
        dec hl
        lddr        ; move from top to bottom
umovedone: exx
        pop bc      ; pop new TOS
        next

;Z SKIP   c-addr u c -- c-addr' u'
;Z                          skip matching chars
; Although SKIP, SCAN, and S= are perhaps not the
; ideal factors of WORD and FIND, they closely
; follow the string operations available on many
; CPUs, and so are easy to implement and fast.
    head skip,4,SKIP,docode
        ld a,c      ; skip character
        exx
        pop bc      ; count
        pop hl      ; address
        ld e,a      ; test for count=0
        ld a,b
        or c
        jr z,skipdone
        ld a,e
skiploop: cpi
        jr nz,skipmis   ; char mismatch: exit
        jp pe,skiploop  ; count not exhausted
        jr skipdone     ; count 0, no mismatch
skipmis: inc bc         ; mismatch!  undo last to
        dec hl          ;  point at mismatch char
skipdone: push hl   ; updated address
```

```
        push bc     ; updated count
        exx
        pop bc      ; TOS in bc
        next

;Z SCAN    c-addr u c -- c-addr' u'
;Z                      find matching char
    head scan,4,SCAN,docode
        ld a,c      ; scan character
        exx
        pop bc      ; count
        pop hl      ; address
        ld e,a      ; test for count=0
        ld a,b
        or c
        jr z,scandone
        ld a,e
        cpir        ; scan 'til match or count=0
        jr nz,scandone  ; no match, BC & HL ok
        inc bc          ; match!  undo last to
        dec hl          ;   point at match char
scandone: push hl   ; updated address
        push bc     ; updated count
        exx
        pop bc      ; TOS in bc
        next

;Z S=    c-addr1 c-addr2 u -- n   string compare
;Z             n<0: s1<s2, n=0: s1=s2, n>0: s1>s2
    head sequal,2,S=,docode
        push bc
        exx
        pop bc      ; count
        pop hl      ; addr2
        pop de      ; addr1
        ld a,b      ; test for count=0
        or c
        jr z,smatch     ; by definition, match!
sloop:  ld a,(de)
        inc de
        cpi
        jr nz,sdiff     ; char mismatch: exit
        jp pe,sloop     ; count not exhausted
smatch: ; count exhausted & no mismatch found
        exx
        ld bc,0         ; bc=0000  (s1=s2)
        jr snext
sdiff:  ; mismatch!  undo last 'cpi' increment
        dec hl          ; point at mismatch char
        cp (hl)         ; set cy if char1 < char2
        sbc a,a         ; propagate cy thru A
        exx
        ld b,a          ; bc=FFFF if cy (s1<s2)
        or 1            ; bc=0001 if ncy (s1>s2)
        ld c,a
snext:  next

*INCLUDE camel80d.azm   ; CPU Dependencies
*INCLUDE camel80h.azm   ; High Level words
lastword EQU link   ; nfa of last word in dict.
enddict EQU $       ; user's code starts here
        END
```